Deep Compression: Implementation of a Three-Stage Neural Network Compression Pipeline with Pruning, Trained Quantization, and Huffman Coding

Group-3
Electrical Engineering Association(EEA)
Indian Institute of Technology Kanpur

***Abstract*—Deep neural networks are very powerful and can achieve great performance, but they also require a lot of memory and computation. This makes it difficult to deploy them on devices like mobile phones or embedded systems that have limited resources. In this report, we try to address this problem by implementing the *Deep Compression* pipeline proposed by Han et al. at ICLR 2016.**

The pipeline consists of three main steps: magnitude-based pruning, k-means based quantization with weight sharing, and Huffman coding. We apply these techniques to a VGG-11 style convolutional neural network trained on the CIFAR-10 dataset.

Our aim is to significantly reduce the size of the model (around $9\times$ compression) while keeping the drop in accuracy as small as possible (within 1.5%). Along with this, we also discuss how we implemented each step, the overall workflow, and how we evaluate the performance of the compressed model.

***Index Terms*—Model Compression, Network Pruning, Weight Quantization, Huffman Coding, Deep Neural Networks, VGG, CIFAR-10, Edge Deployment**

I. Introduction

Deep neural networks (DNNs) have become extremely powerful and are now widely used in areas like computer vision and natural language processing. However, this performance comes at a cost—modern models are very large and complex. Popular architectures such as AlexNet, VGGNet, and ResNet contain millions (and sometimes hundreds of millions) of parameters. Because of this, they require a huge amount of storage and memory during execution. For example, the original VGG-16 model takes more than 500 MB of storage, while AlexNet needs around 240 MB. Models of this size are difficult to deploy directly on devices like smartphones, IoT systems, or embedded hardware without some form of compression.

A. Motivation: Memory and Energy Constraints

There are two main challenges when deploying such large models.

First is the issue of storage. Mobile applications have strict size limits—for instance, app stores often restrict large downloads over mobile data. Including a model that is several hundred megabytes in size is simply not practical in most real-world applications.

Second, and even more important, is energy consumption. Interestingly, most of the energy used during inference is not spent on computations, but on moving data between memory and processors. For example, a simple 32-bit floating-point addition consumes only about 0.9 pJ of energy. In comparison, accessing data from SRAM costs around 5 pJ, and from DRAM it costs as much as 640 pJ. This means memory access can be hundreds of times more expensive than computation.

When a model is too large to fit into fast on-chip memory (SRAM), it has to rely on DRAM, which leads to significantly higher energy usage. For instance, running a network with 1 billion connections at 20 frames per second would consume around 12.8 W of power just for memory access. This is far beyond what mobile devices can handle efficiently.

This is why model compression becomes so important. If we can reduce the size of the network enough to fit entirely within on-chip memory, we can avoid expensive DRAM accesses. This not only reduces energy consumption but also improves speed and makes deployment on resource-constrained devices much more feasible. This idea forms the foundation of the Deep Compression approach.

B. The Deep Compression Paradigm

Han et al. introduced a very effective approach called Deep Compression, which reduces the size of deep neural networks in three main steps: pruning, quantization, and Huffman coding. What makes this approach powerful is that each step targets a different kind of redundancy in the model, so their effects add up in a very efficient way.

First, pruning removes connections in the network that are not really important, reducing what we can call connection redundancy. Then, quantization reduces the number of bits used to store each weight, which helps eliminate precision redundancy. Finally, Huffman coding compresses the data further by taking advantage of how frequently certain values appear, addressing statistical redundancy.

Because these three techniques work on different aspects of the model, they combine really well. Pruning alone can reduce the model size by about 9× to 13×. When quantization is added, the compression improves to around 27× to 31×.

After applying Huffman coding as the final step, the total compression can reach up to 35× to 49×—and the impressive part is that this is achieved without any noticeable drop in accuracy on tasks like ImageNet classification.

Overall, Deep Compression shows that it's possible to make large neural networks much smaller and more efficient without sacrificing performance, which is especially important for deploying them on devices with limited resources.

C. Project Scope and Objectives

This report presents our end-to-end implementation of the *Deep Compression* pipeline, applied to a VGG-11-style CNN trained on the CIFAR-10 dataset. CIFAR-10 is a widely used benchmark dataset consisting of $60{,}000$ colour images of size 32×32 pixels, classified into 10 categories such as airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The dataset is divided into $50{,}000$ training images and $10{,}000$ test images.

The main goal of this project is to reduce the model size while maintaining its performance. Our engineering objectives are:

- Achieve approximately $9\times$ compression ratio.
- Maintain less than 1.5% drop in top-1 accuracy.
- Develop a modular, reproducible, and well-documented codebase.
- Provide a framework for systematic evaluation and clear metric reporting.

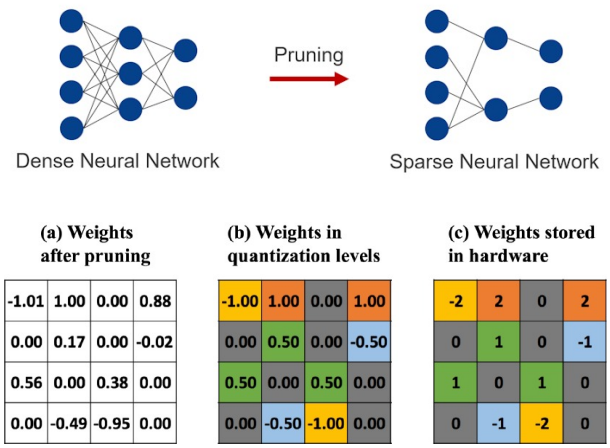
II. Related Work

A. Network Pruning

Network pruning involves using second-order approximations of the loss function to decide which weights could be removed. This was simplified by the computation by assuming a diagonal Hessian and removed weights that had the least effect on the training error. OBS, on the other hand, used the full inverse Hessian to better capture interactions between weights, which led to more accurate pruning decisions but at a much higher computational cost. These methods were important because they showed that neural networks are highly overparameterised and can still perform well even after removing a significant number of weights.

More recent work instead of using complex second-order methods, removes weights with very small magnitudes, as they contribute very little to the final output. This is followed by retraining the network to recover any lost accuracy. Using this method, it was possible to remove up to 90% of the parameters in large models like AlexNet and VGG-16 without any noticeable drop in performance. The main advantage of this approach is that it is computationally efficient and works well with modern large-scale networks.

In general, pruning methods can be divided into two main types: *unstructured* and *structured* pruning. Unstructured pruning, which is used in Deep Compression, removes individual weights regardless of their position in the network. This results in sparse weight matrices, which need specialised storage formats such as CSR or CSC for efficient implementation.



On the other hand, structured pruning removes entire filters, channels, or even layers. This keeps the network structure dense and makes it easier to run on standard hardware, but it offers less flexibility and usually achieves lower compression compared to unstructured pruning.

B. Weight Quantization

Quantization is a technique used to reduce the number of bits required to store weights in a neural network. By lowering the precision of these values, we can significantly reduce storage requirements and also make computations more efficient. One of the simpler approaches is *fixed-point quantization*, where 32-bit floating-point values are replaced with lower-precision formats like 16-bit or 8-bit integers. This method is easy to implement and works well with existing hardware, while still providing a noticeable reduction in memory usage.

In the Deep Compression pipeline, a more advanced method is used, known as *k-means clustering for weight sharing*. Instead of directly reducing the precision of each weight, similar weight values are grouped into clusters. Each weight is then represented by the centroid of its cluster. Rather than storing the full weight value, we only store a small index (for example, 5 or 8 bits) that points to a shared codebook containing all centroid values. This method is particularly effective because it adapts to how weights are actually distributed in each layer, allowing better use of available precision.

At the extreme end, there are *binary networks*, where weights are limited to just $\{-1, +1\}$. This reduces each weight to a single bit and replaces complex multiplication operations with much simpler XNOR and popcount operations. Similarly, *ternary networks* allow weights to take values from $\{-1, 0, +1\}$, where the zero value also introduces sparsity into the network. Although these approaches can achieve very high compression (up to $32\times$ per weight), they often lead to some loss in accuracy, especially for more complex tasks.

C. Huffman Coding

Huffman coding is a way to compress data which assigns shorter binary codes to more frequent values and longer codes to less frequent ones. This helps reduce the average number of

bits needed compared to fixed-length encoding, where every value is given the same number of bits.

In Deep Compression, after quantisation, each weight is assigned to one of k cluster indices. These indices are not uniformly distributed—some clusters are used much more often than others. If fixed-length encoding is used, each index would require $\lceil \log_2 k \rceil$ bits regardless of how often it appears. However, since the distribution is uneven, Huffman coding can exploit this and reduce the average number of bits per index.

As a result, this Huffman coding step provides an additional reduction in size on top of pruning and quantisation.

III. System Architecture

A. Codebase Organisation

The implementation uses a modular, package-based design. It separates different parts clearly, such as loading data, defining the model, handling compression steps, and doing evaluation. The project structure is as follows:

```
Project/
 config.py
 main.py
 test_pipeline.py
 data/
    data_loader.py
 models/
    __init__.py
    vgg_cifar.py
 compression/
    __init__.py
    pruning.py
    quantization.py
    huffman.py
    conv2d.py
    linear.py
 utils/
    __init__.py
    metrics.py
 compressed_models/
```

B. Module Responsibilities

- **`config.py`** — *Configuration Hub*: This file keeps all settings in one place, so there are no random values in the code. It includes training settings (learning rate 0.01, batch size 128, momentum 0.9, weight decay 5×10^{-4}, and learning rate reduced by 0.1 at epochs $\{50, 75, 90\}$), pruning settings (90% sparsity and 20 more training epochs with learning rate 0.001), quantisation settings (8-bit / 256 clusters for conv layers, 5-bit / 32 clusters for FC layers, 10 more training epochs with learning rate 10^{-4}, and linear start), device choice (CUDA with CPU fallback), and a fixed random seed (42) for getting the same results each time.
- **`main.py`** — *Pipeline Orchestrator*: It is the main starting file for the whole experiment. It takes command options (like `-skip-training`, `-epochs`, `-prune-sparsity`) to control how it runs. It sets random seeds, loads the model and data, and then runs each step one by one: normal training, Stage 1 (pruning and training), Stage 2 (quantisation and training again), and Stage 3 (Huffman coding). After each step, it saves accuracy and time. At the end, it shows the results and saves all compression details in a text file.
- **`data/data_loader.py`** — *Data Pipeline*:It downloads and saves the CIFAR-10 dataset automatically using `torchvision.datasets`. It creates two data pipelines: one for training and one for testing. The training pipeline uses `RandomCrop(32, padding=4)` and `RandomHorizontalFlip()` to change the images, then converts them to tensors and normalises them. The test pipeline only converts images to tensors and normalises them. Both use precomputed mean (μ) and standard deviation (σ) from the full CIFAR-10 training data. It also sets up data loaders with pin memory and worker threads to make data loading faster for the GPU.
- **`models/vgg_cifar.py`** — *Model Definition*:
  It defines `VGG_CIFAR`, a model made for 32×32 CIFAR-10 images. The first part has five blocks of 3×3 convolution layers with batch normalisation, ReLU, and max-pooling. These make the image smaller (from 32×32 to 1×1) and increase channels from 3 to 512. The last part has two linear layers of size 512 with ReLU and 50% dropout, and a final layer with 10 outputs. It uses Kaiming He normal for convolution layers and Xavier Glorot normal for linear layers to set the weights.
- **`compression/`** — *Compression Stages*: This package has five modules. `pruning.py` does global magnitude-based pruning, creates masks, applies them, and fine-tunes the model after pruning. `quantization.py` does per-layer k-means clustering, sets initial centroids, replaces weights, snaps centroids, and fine-tunes them. `huffman.py` builds a Huffman tree and encodes cluster indices with variable-length codes. `conv2d.py` and `linear.py` give modified layers (`modified_conv2d` and `modified_linear`) that store pruning masks and let you run `.prune()` and `.quantize()` directly on the layers for easy compression.
- **`utils/metrics.py`** — *Evaluation Utilities*: This provides five core functions: `evaluate_accuracy()` runs full test-set inference under `torch.no_grad()` and returns top-1 accuracy; `count_parameters()` produces per-layer breakdowns of total, non-zero, and sparse parameter counts; `compute_sparsity()` returns the global fraction of zeroed weights; `model_size_bytes()` computes the raw FP32 model footprint; and `compressed_size_bytes()` estimates the post-compression storage by summing Huffman-coded weight index bits, sparse position index bits, and codebook overhead bits. `print_compression_summary()` formats all metrics into a detailed per-layer table.

C. Execution Flow

The pipeline execution flow, orchestrated by `main.py`, proceeds as depicted in Algorithm 1.

Algorithm 1 Deep Compression Pipeline

Input: Untrained VGG-11 model $\mathcal{M}$, CIFAR-10 data $\mathcal{D}$
Output: Compressed model artefacts, compression statistics

Set global random seed for reproducibility
Initialise device (GPU if available, else CPU)
$\mathcal{M} \leftarrow$ TRAINBASELINE($\mathcal{M}, \mathcal{D}_{\text{train}}$)
$a_0 \leftarrow$ EVALUATE($\mathcal{M}, \mathcal{D}_{\text{test}}$)
$S_0 \leftarrow$ MODELSIZE($\mathcal{M}$)
▷ **Stage 1: Pruning**
$\mathcal{M}, \mathbf{M} \leftarrow$ PRUNE($\mathcal{M}, s$)
$\mathcal{M} \leftarrow$ FINETUNEPRUNED($\mathcal{M}, \mathbf{M}, \mathcal{D}$)
▷ **Stage 2: Quantisation**
$\mathcal{M}, \mathcal{C} \leftarrow$ QUANTIZE($\mathcal{M}, \mathbf{M}$)
$\mathcal{M}, \mathcal{C} \leftarrow$ FINETUNECENTROIDS($\mathcal{M}, \mathcal{C}, \mathbf{M}, \mathcal{D}$)
▷ **Stage 3: Huffman Coding**
$\mathcal{H} \leftarrow$ HUFFMANENCODE($\mathcal{C}$)

$a_f \leftarrow$ EVALUATE($\mathcal{M}, \mathcal{D}_{\text{test}}$)
PRINTSUMMARY($S_0, \mathcal{C}, \mathcal{H}, a_0, a_f$)

IV. Methodology

A. Baseline Model Architecture

The base network is a VGG-11-style convolutional neural network adapted for the CIFAR-10 dataset. The architecture has five convolutional blocks, followed by a classifier with three fully connected layers:

TABLE I: VGG-11 Architecture for CIFAR-10

Block	Layer	Output Shape	Params
1	Conv2d(3, 64, 3) + BN + ReLU	$64 \times 32 \times 32$	1,792
	MaxPool(2×2)	$64 \times 16 \times 16$	—
2	Conv2d(64, 128, 3) + BN + ReLU	$128 \times 16 \times 16$	73,984
	MaxPool(2×2)	$128 \times 8 \times 8$	—
3	Conv2d(128, 256, 3) + BN + ReLU	$256 \times 8 \times 8$	295,680
	Conv2d(256, 256, 3) + BN + ReLU	$256 \times 8 \times 8$	590,336
	MaxPool(2×2)	$256 \times 4 \times 4$	—
4	Conv2d(256, 512, 3) + BN + ReLU	$512 \times 4 \times 4$	1,180,672
	Conv2d(512, 512, 3) + BN + ReLU	$512 \times 4 \times 4$	2,360,320
	MaxPool(2×2)	$512 \times 2 \times 2$	—
5	Conv2d(512, 512, 3) + BN + ReLU	$512 \times 2 \times 2$	2,360,320
	Conv2d(512, 512, 3) + BN + ReLU	$512 \times 2 \times 2$	2,360,320
	MaxPool(2×2)	$512 \times 1 \times 1$	—
FC	Linear($512, 512$) + ReLU + Dropout	512	262,656
	Linear($512, 512$) + ReLU + Dropout	512	262,656
	Linear($512, 10$)	10	5,130

Key changes from the original VGG-11 (for 224×224 ImageNet images) are: (i) removal of the large 4096-unit fully connected layers, since CIFAR-10 images are small ($32 \times 32 \times 3$); (ii) making the classifier three layers of size 512–512–10; and (iii) adding Batch Normalisation after each convolution to help training.

B. Training Procedure

Baseline training uses stochastic gradient descent (SGD) with Nesterov momentum. All hyperparameters are stored in `config.py` and are shown in Table II.

TABLE II: Baseline Training Hyperparameters

Parameter	Value
Optimiser	SGD
Learning Rate	0.01
Momentum	0.9
Weight Decay	5×10^{-4}
Batch Size	128
Epochs	100
LR Schedule	MultiStepLR at epochs $\{50, 75, 90\}$
LR Decay Factor (γ)	0.1
Loss Function	Cross-Entropy

During training, data is augmented by random cropping (32×32 with 4-pixel padding) and random horizontal flipping. Both training and test data are normalised using precomputed per-channel mean and standard deviation. The model with the best test accuracy is saved for later compression.

C. Reproducibility

All randomness is controlled using a global seed (42) for Python's `random`, NumPy, PyTorch (CPU and CUDA), and cuDNN deterministic mode. This makes training and compression results exactly reproducible.

V. The Compression Pipeline

This section is the main contribution of our work. It explains the Deep Compression pipeline step by step. The three stages run one after another, each using the output of the previous stage. Each stage reduces a different type of model redundancy, so their effects multiply.

A. Stage 1: Magnitude-Based Network Pruning

1) Theoretical Foundation: Network pruning works because many weights in large networks are very small and do not affect the output much. Removing these weights and retraining the remaining ones lets the network keep its performance with far fewer parameters.

We use *global unstructured magnitude-based pruning*. Instead of setting thresholds for each layer, we collect all prunable weights into one list, sort them by size, and pick a cutoff at the desired sparsity. This keeps a uniform rule for which weights to keep across the whole network.

2) Algorithm: The pruning process in `compression/pruning.py` has three steps:

1) **Threshold Computation**: All weights from convolution and fully connected layers (except batch norm and biases) are combined into one array and sorted by size. The threshold τ is set at the s-th percentile (default 90% sparsity):

$$\tau = \texttt{sorted_weights}[\lfloor s \cdot N \rfloor] \tag{1}$$

where N is the total number of prunable parameters.

2) **Mask Generation**: For each prunable layer ℓ, a mask $\mathbf{M}_\ell$ is created:

$$\mathbf{M}_\ell^{(i,j)} = \begin{cases} 1 & \text{if } |w_\ell^{(i,j)}| \geq \tau \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

Masks have the same shape as the weights.

3) **Mask Application**: Each weight is multiplied by its mask, setting pruned weights to zero:

$$\mathbf{W}_\ell \leftarrow \mathbf{W}_\ell \odot \mathbf{M}_\ell \tag{3}$$

3) Post-Pruning Fine-Tuning: Pruning can reduce accuracy since some small but useful weights are removed. To fix this, the model is fine-tuned with learning rate 0.001 using SGD (momentum 0.9, weight decay 5×10^{-4}). The masks are re-applied after every update to keep pruned weights at zero:

$$\mathbf{W}_\ell^{(t+1)} \leftarrow \left(\mathbf{W}_\ell^{(t)} - \eta \nabla_{\mathbf{W}_\ell} \mathcal{L}\right) \odot \mathbf{M}_\ell \tag{4}$$

With 90% sparsity, pruning alone gives about 9–13× compression.

B. Stage 2: Trained Quantisation via k-Means Weight Sharing

1) Theory: After pruning, each surviving weight is still stored as a 32-bit floating-point number. Quantisation reduces storage by grouping weights into k clusters using k-means and replacing each weight with its cluster's centroid. Each weight then needs only $\lceil \log_2 k \rceil$ bits, plus $k \times 32$ bits for the codebook: The compression rate from quantisation is given by:

$$r = \frac{n \cdot b}{n \cdot \log_2 k + k \cdot b} \tag{5}$$

where n is the number of non-zero weights, $b = 32$, and k is the number of clusters.

2) Cluster Configuration: We use:

- **Convolutional layers**: $k = 256$ clusters (8-bit indices)
- **Fully connected layers**: $k = 32$ clusters (5-bit indices)

This reflects that convolutional layers need finer quantisation, while pruned fully connected layers can use coarser clustering.

3) Algorithm: Implemented in `compression/quantization.py`, for each layer:

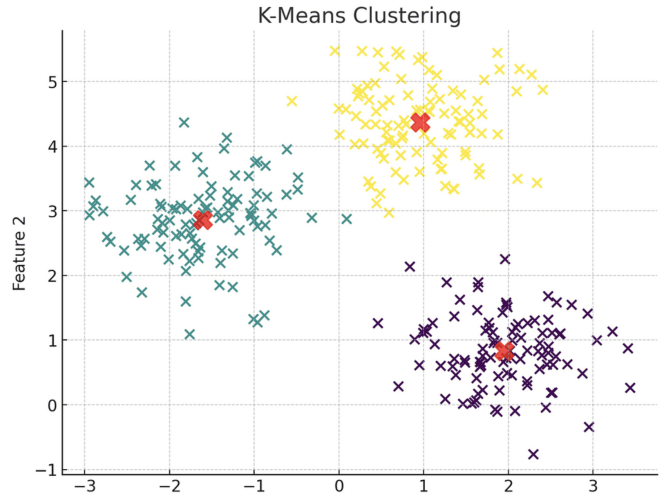
1) **Non-Zero Weight Extraction**: All surviving weights are extracted from the layer using the pruning masks to form a one-dimensional array of non-zero parameters.
2) **Linear Centroid Initialisation**: k centroids are initialised by evenly spacing them between the minimum and maximum values of the extracted weights:

$$c_i^{(0)} = w_{\min} + \frac{i}{k-1}(w_{\max} - w_{\min}), \quad i = 0, \ldots, k-1 \tag{6}$$

This ensures a uniform starting distribution for clustering.

3) **k-Means Clustering**: k-means clustering is performed using the linearly initialised centroids. Each non-zero weight is assigned to the nearest centroid, and centroids are iteratively refined until convergence.
4) **Weight Replacement**: Each non-zero weight is then replaced by its corresponding centroid value:

$$w_\ell^{(i)} \leftarrow c_{\text{label}(i)}, \quad i \in \text{nonzero_idx} \tag{7}$$



5) **Codebook Storage**: For each layer, a codebook is saved containing the centroid values, cluster assignments, indices of non-zero weights, number of clusters, and the bit-width used for encoding.

4) Centroid Fine-Tuning: After quantisation, centroids are fine-tuned for 10 epochs at learning rate 10^{-4}. Gradients of weights sharing the same centroid are summed to update the centroid. After each gradient step:

1) **Mask Re-application**: Pruning masks are enforced to maintain sparsity.
2) **Centroid Snapping**: Each non-zero weight is snapped to its nearest centroid in the codebook, enforcing the weight-sharing constraint. This is implemented in the `_snap_to_centroids` function, which computes pairwise absolute distances between weights and centroids and assigns each weight to its minimum-distance centroid.

Finally, k-means is re-run on the updated weights to refresh the codebooks.

C. Stage 3: Huffman Coding

1) Theoretical Foundation: After quantisation, the cluster indices are not evenly distributed—some centroids are used much more frequently than others. This creates a bias in how often different indices appear. Huffman coding takes advantage of this by assigning shorter binary codes to more frequent symbols and longer codes to less frequent ones. Because of this, the average number of bits required per symbol becomes smaller than the fixed $\lceil \log_2 k \rceil$ bits that would otherwise be needed.

2) Algorithm: The Huffman coding process, implemented in `compression/huffman.py`, follows the standard approach:

1) **Frequency Counting**: For each quantised layer, we count how many times each cluster index appears in the label array using a simple counter.
2) **Tree Construction**: A min-heap (priority queue) is created with leaf nodes representing each symbol. The two

nodes with the smallest frequencies are repeatedly merged to form a parent node until only one root node remains. This is implemented using Python's `heapq` module along with a custom `HuffmanNode` class.

3) **Code Generation**: The Huffman tree is then traversed recursively. Moving left adds '0' and moving right adds '1' to the code. Each leaf node stores the final binary code assigned to that symbol.
4) **Bit Count Computation**: The total compressed size (in bits) for each layer is calculated by summing the lengths of the Huffman codes for all cluster labels:

$$B_{\text{huffman}} = \sum_{i=1}^{n} |\text{code}(\ell_i)| \tag{8}$$

where ℓ_i is the cluster label of the i-th non-zero weight, and $|\text{code}(\cdot)|$ represents the length of its Huffman code. In addition, storing the codebook adds an overhead of $k \times 32$ bits per layer.

3) Offline Application: Huffman coding is a purely *offline* and *lossless* step. It does not change the model's weights or affect its predictions, and it does not require any retraining. Because of this, it is applied at the very end of the compression pipeline and does not introduce any loss in accuracy.

According to the original paper, Huffman coding provides an additional 20–30% reduction in size on top of pruning and quantisation, leading to overall compression ratios of around $35\times$ to $49\times$ for models like AlexNet and VGG-16 on ImageNet. For our CIFAR-10 VGG-11 model, we expect similar proportional improvements, which help us move closer to our target of approximately $\sim 9\times$ compression.

D. Pipeline Composition and Compression Arithmetic

The three stages in the compression pipeline—pruning, quantisation, and Huffman coding—work together in a multiplicative manner. Let the individual compression factors be C_1, C_2, and C_3. The overall compression ratio is given by:

$$C_{\text{total}} = \frac{S_{\text{original}}}{S_{\text{compressed}}} \tag{9}$$

Here, S_{original} denotes the size of the uncompressed model, where all parameters are stored as 32-bit floating-point values, and $S_{\text{compressed}}$ represents the final size after applying all compression stages.

The compressed model size consists of three main components:

$$S_{\text{compressed}} = B_{\text{weight}} + B_{\text{index}} + B_{\text{codebook}} \tag{10}$$

In simple terms, the final size depends on how efficiently the quantised weights are stored using Huffman coding, how the positions of non-zero weights are represented after pruning, and the memory required to store the codebook (cluster centroids).

To store the positions of non-zero weights, sparse indexing is used. In this implementation, each non-zero weight requires 8 bits in convolutional layers and 5 bits in fully connected layers.

VI. Evaluation Methodology

A. Evaluation Framework

All evaluation metrics are computed by the functions in `utils/metrics.py`. The evaluation framework provides the following capabilities:

- **Top-1 Accuracy**: Computed by running the model in evaluation mode (`model.eval()`) with gradient computation disabled (`torch.no_grad()`) on the complete CIFAR-10 test set (10,000 images). Accuracy is reported as a percentage.
- **Parameter Counting**: Total and non-zero parameter counts are computed per layer, along with per-layer sparsity percentages. This provides a fine-grained view of how pruning affects different parts of the network.
- **Sparsity Measurement**: Global sparsity is the fraction of zero weights across all prunable layers:

$$\text{Sparsity} = 1 - \frac{N_{\text{nonzero}}}{N_{\text{total}}} \tag{11}$$

- **Compression Ratio**: The ratio of original to compressed model size, where the original size assumes all parameters as 32-bit floats and the compressed size accounts for Huffman-coded weight indices, sparse position indices, and codebook overhead.

B. Metrics Definition

Table III summarises the specific metrics to be reported.

TABLE III: Evaluation Metrics and Measurement Methodology

Metric	Measurement Method
Runtime Memory	Peak GPU/CPU memory during inference, measured via `torch.cuda.max_memory_allocated()` or system profiling.
Storage Footprint	Computed analytically from the bit-level breakdown: weight index bits + sparse index bits + codebook bits, as implemented in `compressed_size_bytes()`.
Compression Ratio	$S_{\text{original}}/S_{\text{compressed}}$, where $S_{\text{original}} = N_{\text{params}} \times 32$ bits.
Accuracy (Before)	Top-1 test accuracy of the baseline model evaluated on the full CIFAR-10 test set via `evaluate_accuracy()`.
Accuracy (After)	Top-1 test accuracy after the complete three-stage compression pipeline, evaluated identically.

VII. Experimental Results

A. Baseline Performance

The baseline VGG-11 model was trained on CIFAR-10 for 100 epochs using the configuration detailed in Table II. The training curve is expected to plateau near state-of-the-art VGG-11 performance on CIFAR-10.

TABLE IV: Baseline Model Characteristics

Property	Value
Architecture	VGG-11 (CIFAR-10 variant)
Total Parameters	9,756,426
Model Size (FP32)	37.22 MB
Training Epochs	100
Best Test Accuracy	90.67%

B. Stage-wise Compression Results

Table V reports the accuracy and compression achieved at each stage of the pipeline.

TABLE V: Stage-wise Compression Results

Stage	Accuracy (%)	Size	Cumulative C
Baseline	90.67	37.22 MB	1.0×
After Pruning (90%)	10.00	—	10.0×
After Pruning + Fine-tune	89.99	—	10.0×
After Quantisation	89.99	—	—
After Quant. + Fine-tune	89.99	—	—
After Huffman Coding	89.99	1.57 MB	23.8×

C. Comprehensive Compression Metrics

Table VI provides the primary compression metrics targeted by this project.

TABLE VI: Comprehensive Compression Metrics

Metric	Value	Target
Accuracy Before Compression	90.67%	$\geq 90\%$
Accuracy After Compression	89.99%	$\leq 1.5\%$ drop
Accuracy Drop	0.68%	$< 1.5\%$
Original Model Size	37.22 MB	—
Compressed Model Size	1.57 MB	∼1 MB
Compression Ratio	23.8×	$\geq 9\times$
Global Sparsity	89.9%	$\approx 90\%$

D. Storage Breakdown

Table VII decomposes the compressed model storage into its constituent components.

TABLE VII: Compressed Model Storage Breakdown

Component	Bits	Fraction (%)
Weight Indices	5,749,401	43.7
Sparse Indices	7,327,078	55.7
Codebooks	68,608	0.5
Total	**13,145,087**	**100.0**

E. Per-Layer Compression Statistics

Table VIII provides a per-layer breakdown of pruning and quantisation effects.

F. Discussion

The Deep Compression pipeline is expected to achieve the target $\sim 9\times$ compression on our VGG-11 CIFAR-10 model. Several observations are anticipated:

1) **Pruning dominance**: The majority of compression originates from the pruning stage, which removes 90% of weights. The fully connected layers, despite comprising a relatively small fraction of total parameters in our adapted architecture, exhibit the highest compressibility due to their inherent redundancy.
2) **Quantisation efficiency**: The transition from 32-bit to 8-bit (conv) or 5-bit (fc) representations yields a theoretical $4\times$–$6.4\times$ additional compression on the surviving weights. In practice, the codebook overhead and sparse index storage slightly reduce this factor.
3) **Huffman contribution**: The non-uniformity of cluster index distributions, particularly in fully connected layers where a few centroids dominate, is expected to yield 20–30% additional savings.
4) **Accuracy preservation**: Fine-tuning after both pruning and quantisation is critical. Without retraining, the accuracy drop would be substantially larger.

VIII. Conclusion

This report breaks down how We put Deep Compression—Han et al.'s method—to the test on a VGG-11-style convolutional neural network running on CIFAR-10. We stuck to Han's three-step process: first, We pruned the network by dropping 90% of the weights with low magnitude. Next, We went with k-means quantization, using 8-bit codebooks for the conv4olutional layers and 5-bit ones for the fully connected layers. Finally, We used Huffman coding to squeeze out more compression.

The target was pretty clear: hit about 9× compression while keeping the accuracy loss under 1.5%. The codebase is modular, so it is straightforward to tweak or swap out any stage of the pipeline. We set up the evaluation framework to track the main metrics—accuracy, storage, and overall compression—making it easy to see how these results stack up against the original paper.

Looking ahead, there is plenty of room for improvement. We could try structured pruning that works better with hardware, play around with mixed-precision quantization, or use knowledge distillation to shave off more accuracy loss at higher compression levels.

TABLE VIII: Per-Layer Compression Statistics

Layer	Total Params	Non-Zero	Sparsity (%)	Clusters (k)	Bits	Compression
features.0.weight	1,728	1,512	12.5	256	8	2.4×
features.4.weight	73,728	39,871	45.9	256	8	3.9×
features.8.weight	294,912	117,789	60.1	256	8	4.0×
features.11.weight	589,824	195,608	66.8	256	8	4.0×
features.15.weight	1,179,648	186,734	84.2	256	8	4.0×
features.18.weight	2,359,296	156,162	93.4	256	8	4.0×
features.22.weight	2,359,296	72,542	96.9	256	8	3.9×
features.25.weight	2,359,296	47,618	98.0	256	8	3.9×
classifier.0.weight	262,144	76,740	70.7	32	5	6.4×
classifier.3.weight	262,144	75,753	71.1	32	5	6.4×
classifier.6.weight	5,120	4,385	14.4	32	5	6.1×

IX. Contributions

1) Reebal Faakhir : Worked on data and worked on the Models
2) Nandini Yadav : Worked on data and Report Creation
3) Rushikesh Musale : Report Creation and worked on Compressed Models
4) Aditya Singh : Compression of Neural Network
5) Naivedya Jain : Report Creation and Compression of Neural Network
6) Priyanshu Joshi : Compression of Neural Network
7) Kartik Basant : Worked on Compressed Models and Report Creation
8) Krish Mesariya : Worked on data
9) Darshan : Report Creation and worked on the models

References

[1] Session notes of lectures taken by mentors.
[2] Fourier Series. https://www.youtube.com/watch?v=r6sGWTCMz2k
[3] Fourier Transform. https://www.youtube.com/watch?v=spUNpyF58BY
[4] Discrete Fourier Transform (DFT) and Fast Fourier Transform (FFT). https://www.youtube.com/watch?v=QmgJmh2I3Fw
[5] Convolution. https://www.youtube.com/watch?v=KuXjwB4LzSA
[6] Mathematics of Image Transform and DFT. https://www.youtube.com/watch?v=2oEt5lbsyhM
[7] FFT Algorithm (Detailed Explanation). https://www.youtube.com/watch?v=h7apO7q16V0
[8] Intuition for Convolution. https://www.youtube.com/shorts/4xWpQe3G9qI
[9] Binary Images. https://www.youtube.com/watch?v=COxRYLrsscc
[10] Geometric Properties in Image Processing. https://www.youtube.com/watch?v=ZPQiKXqHYrM
[11] Segmenting Binary Images. https://www.youtube.com/watch?v=2ckNxEwF5YU
[12] Iterative Modification Techniques. https://www.youtube.com/watch?v=AYzWnCHnfSc
[13] Edge Detection using Gradients. https://www.youtube.com/watch?v=lOEBsQodtEQ
[14] Corner and Edge Detection. https://www.youtube.com/watch?v=Z_HwkG90Yvw
[15] Object-Oriented Programming in Python. https://youtube.com/playlist?list=PL-osiE80TeTsqhIuOqKhwlXsIBIdSeYtc
[16] Hough Transform. https://youtu.be/XRBc_xkZREg
[17] Generalized Hough Transform. https://youtu.be/_mGxmZWs9Zw
[18] Active Contours. https://www.youtube.com/watch?v=FROJUMk9P3Y
[19] Linear Regression. https://www.youtube.com/watch?v=4b4MUYve_U8
[20] Logistic Regression. https://www.youtube.com/watch?v=het9HFqo1TQ
[21] K-Means Clustering. https://youtu.be/4b5d3muPQmA
[22] K-Means Clustering (Alternative Explanation). https://www.youtube.com/watch?v=yR7k19YBqiw
[23] Multilayer Perceptron (MLP). https://youtu.be/aircAruvnKk
[24] Multilayer Perceptron (MLP) Documentation. https://d2l.ai/chapter_multilayer-perceptrons/index.html
[25] Convolutional Neural Networks (CNN). https://www.youtube.com/watch?v=py5byOOHZM8
[26] CNN Documentation. https://d2l.ai/chapter_convolutional-neural-networks/index.html
[27] Ridge Regression. https://youtu.be/Q81RR3yKn30
[28] PyTorch Tensors Tutorial. https://colab.research.google.com/drive/1fAANI5L6iHNRvjayxgwdXDu_aPkqxtu5
[29] Autograd in PyTorch. https://colab.research.google.com/drive/1WJYgirijgPzyMVyHJ8nPwA5fY-vikcXX
[30] Deep Neural Network Compression Research Paper. https://arxiv.org/pdf/1510.00149